Authentication Methods

BCS2420

Dr. Ashish Sai

ashish.sai@maastrichtuniversity.nl

User Authentication

User Authentication

Process of using **supporting evidence** to **corroborate** an asserted ***identity***.

Authentication vs Identification:

- **Authentication**: One-to-one test to verify identity using an asserted identity (e.g., username and password).
- **Identification**: One-to-many test to establish identity from available information without an asserted identity (e.g., facial recognition in a crowd).

Purpose

- **End-goals**:
  - Authentication: Verify identity.
  - Authorization: Determine access to privileges or resources.

Example: Password required to install or upgrade software on a device.

Password Authentication

Basic Concept

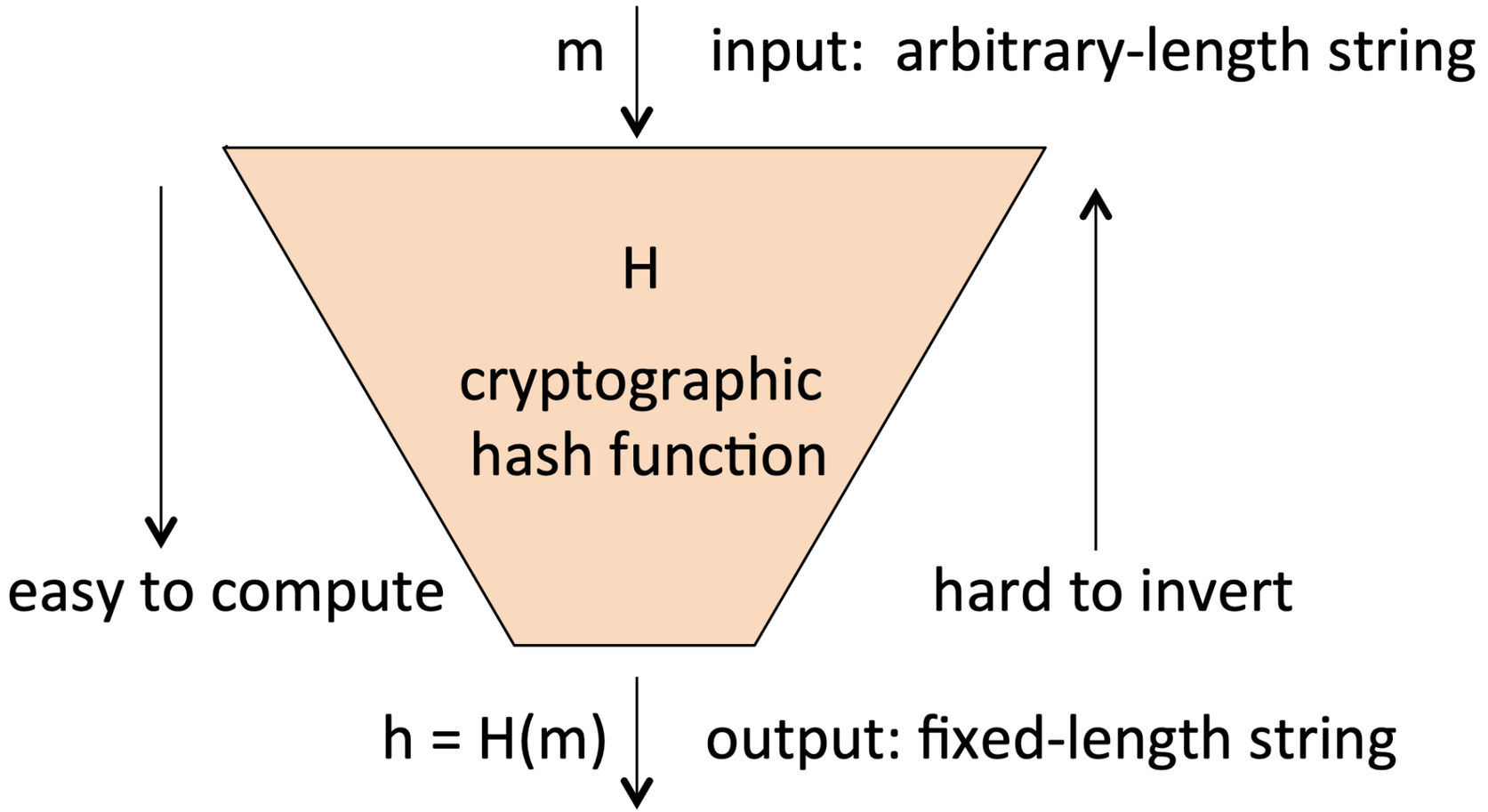
- User enters a `username` and `password` to access an account.
  - System verifies the password against stored information.
  - A correct password match **does not** ensure that whoever entered it is the authorized user; it only indicates knowledge of the password.

Storing Passwords

- **Cleartext Storage**: Risky; exposes all passwords if file is stolen.
- **Hash Storage**: Store password **hashes** instead of cleartext passwords using a one-way hash function.

Example

- Use of one-way hash function to store passwords securely.


m
input: arbitrary-length string
H
cryptographic
hash function
easy to compute
hard to invert
h = H(m)
output: fixed-length string

Approaches to Defeat Password Authentication

Pre-computed Dictionary Attack

Attack Steps:

1. Create a list of candidate passwords.
2. Compute hashes for each password.
3. Compare stolen password hashes with precomputed hashes.

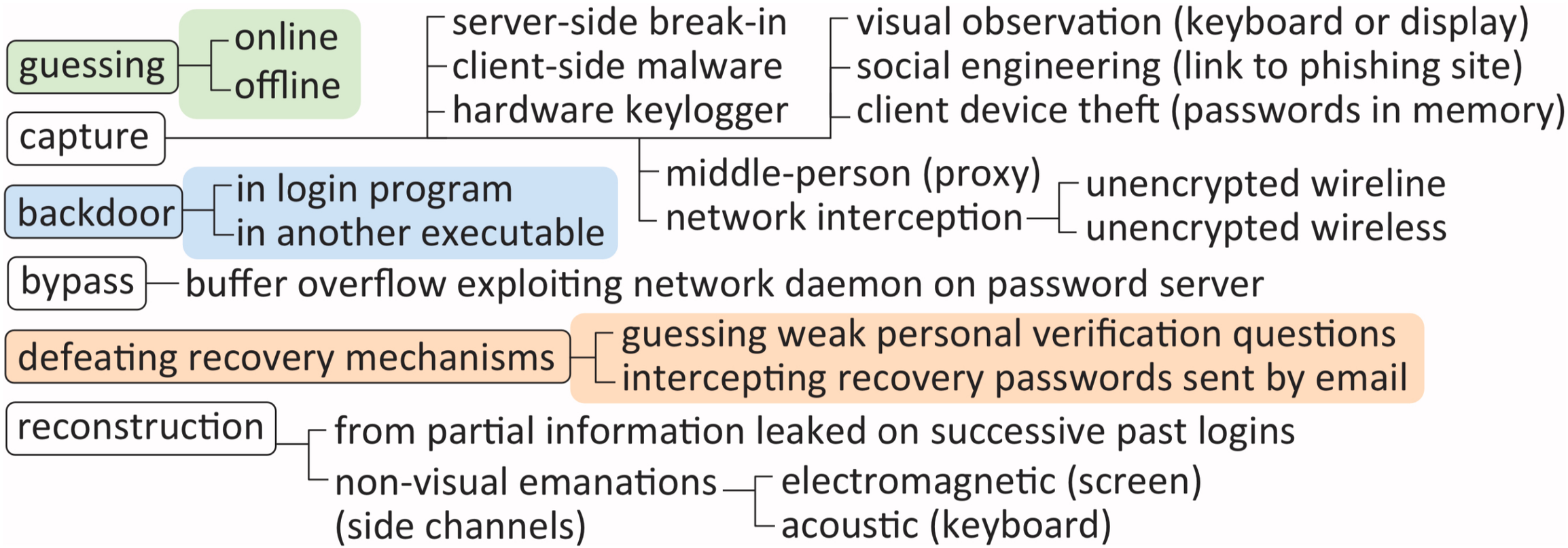
Defense: Use salts [1] and strong hashing algorithms to mitigate risk.

1. A salt is a random value added to data (e.g., passwords) before hashing to prevent attacks like dictionary or rainbow table attacks. ↩

Categories of Password-guessing Attacks

1. **Online Password Guessing**
   - Guesses are sent to the **legitimate server**.
   - The server indicates whether the attempt is correct or not.

2. **Offline Password Guessing**
   - Attacker has acquired the **system password hash file**.
   - No per-guess online interaction is needed.
   - The number of guesses is limited only by computational resources available to the attacker.


guessing
online
offline
capture
server-side break-in
client-side malware
hardware keylogger
visual observation (keyboard or display)
social engineering (link to phishing site)
client device theft (passwords in memory)
middle-person (proxy)
network interception
unencrypted wireline
unencrypted wireless
backdoor
in login program
in another executable
bypass
buffer overflow exploiting network daemon on password server
defeating recovery mechanisms
guessing weak personal verification questions
intercepting recovery passwords sent by email
reconstruction
from partial information leaked on successive past logins
non-visual emanations (side channels)
electromagnetic (screen)
acoustic (keyboard)

Online Password Guessing and Rate-limiting

- Defensive Tactics
  - **Rate-limit guesses**: Throttle guesses across fixed time windows.
  - **Lockout mechanism**: Temporarily **lock** accounts after a certain number of failed attempts.
  - **Example**: Doubling response time after successive incorrect logins: 1s, 2s, 4s, etc.

Offline Password Guessing

- Assumptions
  - Attacker has acquired the password hash file.
  - Hash file provides verifiable text, allowing **correctness** tests without server interaction.
- Defense Mechanisms
  - **Strong Password Policies**: Encourage the use of complex passwords.
  - **Hashing Algorithms**: Use strong, computationally intensive hash functions.
  - **Salting**: Add unique, random values to passwords before hashing.

A modest number of modern GPUs can guess passwords at 400 billion guesses per second 🤯.

Specialized Password-Hashing Functions

- Designed to **resist** GPU and parallel attacks.
- Examples: Argon2, bcrypt, scrypt.

Argon2

Winner of the Password Hashing Competition (PHC) [1], designed to resist parallel attacks on modern hardware.

1. https://www.password-hashing.net ↩

Iterated Hashing (Password Stretching)

- **Concept**: Hashing a password *multiple times* to increase the computational effort required for guessing.
- **Process**:
  1. Hash the password once with a hash function `H`.
  2. Hash the result again, and repeat this process `d` times.
     Stored value $= H^d(p_i)$
- **Example**:
  - For `d = 1000`, it slows down attacks by a factor of 1000.
  - The legitimate server must also compute the iterated hash!

Password Salting

Definition: Adding a unique, random value (salt) to each password before hashing to prevent pre-computed dictionary attacks.

– **Process**:
  1. Select a random salt s_i.
  2. Store the pair $(userid, s_i, H(p_i, s_i))$.

$$h_i = H(p_i \| s_i)$$

Benefit: Prevents attackers from using precomputed tables of hashes.

Pepper (Secret Salt)

- **Concept**: A secret salt not stored, designed to slow down attacks.
- **Process**:
  1. Choose a random value r_i.
  2. Store the secret-salted hash $H(p_i, r_i)$.
  3. Erase r_i.

     Verification : Sequentially try all values r^*
- **Benefit**: Slows down attacks significantly, especially if combined with regular salt.

System-assigned Passwords and Brute-force Guessing

Concept

- **System-assigned Passwords**: Maximizing difficulty of guessing by randomly selecting each character.
- **Password Space**: For an `n`-character password from an alphabet of `b` characters:

$$\text{Password Space} = b^n$$

Probability of Guessing Success

$$q = \frac{GT}{R} \text{ for } GT \leq R$$

- `G`: Number of guesses per unit time.
- `T`: Number of units of time.
- `R` = b^n: Size of the password space.

Example Calculation

– For `n = 10` and `b = 95`:

$$q = \frac{(10^{11})(3.154 \times 10^{7})}{6 \times 10^{19}} = 0.05257$$

User-chosen Passwords and Skewed Distributions

Concept

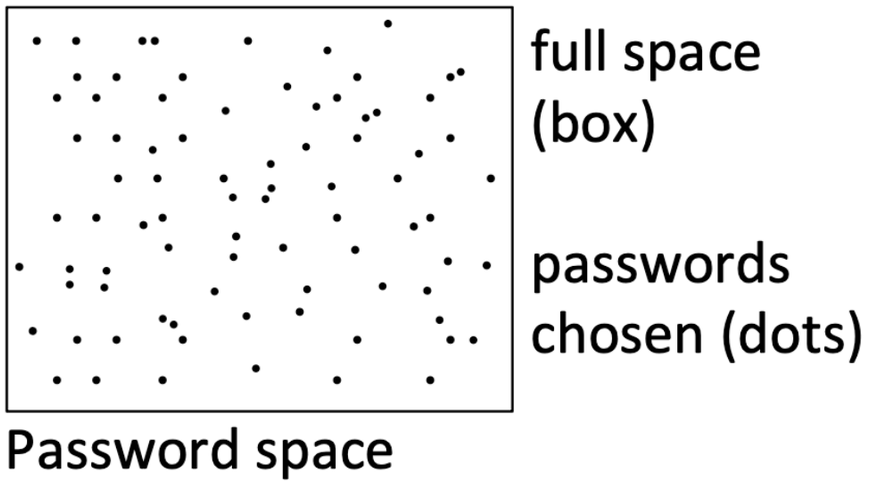
- **Skewed Distributions**: Some passwords are much more popular than others.
- **Attack Strategy**: Attackers try more popular passwords first.

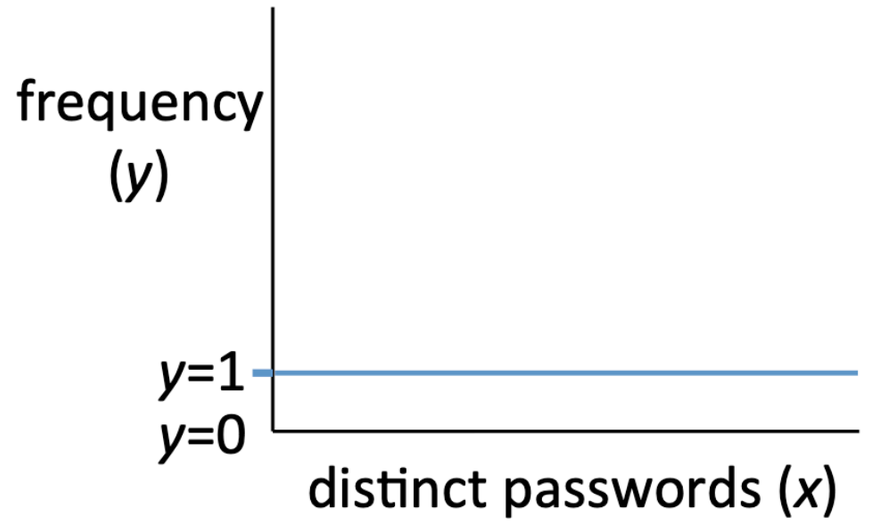
Example

- Attackers leveraging empirical password databases [1] and heuristic means.

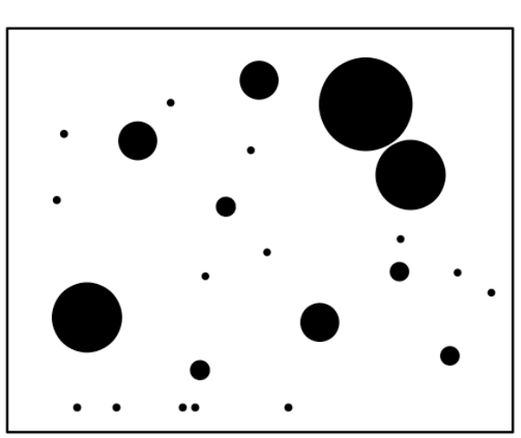
1. An empirical password database is a collection of real-world passwords gathered from data breaches, used to analyze trends and inform attack strategies. ↩

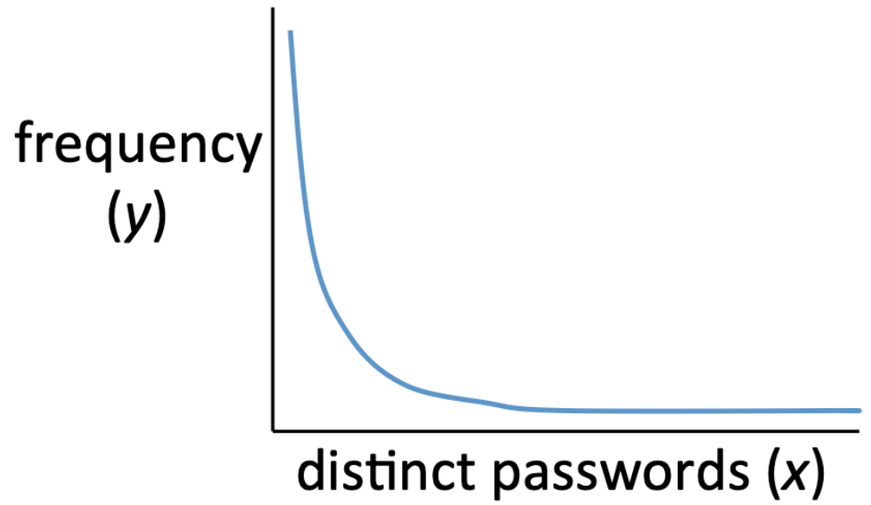
(a) What we want: randomly distributed passwords





(b) What we get: predictable clustering, highly skewed distribution





Password Denylists and Proactive Password Cracking

- **Password Denylists**: Lists of most-popular passwords to disallow at the time of user selection.
- **Proactive Password Cracking**: Systems attempt to crack their own users' passwords and notify users to change easily cracked passwords.

Heuristic Password-cracking Tools

- **Common Tools**: JohnTheRipper [1], Hashcat [2].
- **Techniques**: Use pre-computed tables or hash password guesses on-the-fly with mangling rules.

1. https://github.com/openwall/john ↩
2. https://github.com/hashcat/hashcat ↩

Defensive Measures

Defensive measure	Primary attack addressed	Notes
rate-limiting	online guessing	some methods result in user lockout
denylisting	online guessing	disallows worst passwords
salt	pre-computed dictionary	increases cost of generic attacks
iterated hashing	offline guessing	combine with salting
pepper	offline guessing	alternative to iterated hashing
MAC on password	offline guessing	stolen hash file no longer useful

Password Composition Policies

Best Practices:

- Avoid common passwords.
- Use a combination of **uppercase**, **lowercase**, **digits**, and **special characters**.
- Change passwords regularly.

Example: NIST SP 800-63B [1]

U.S. government password guidelines:

- Use password **denylists** to rule out common, highly predictable passwords.
- Mandate **rate-limiting** to throttle online guessing.
- Recommend against composition rules.
- Mandate secure password storage methods.

1. https://pages.nist.gov/800-63-3/sp800-63b.html ↩

Password Managers

- **Store** and **retrieve** passwords to reduce cognitive burden.
- User remembers **one master password**.

Security Benefits:

- Allows use of strong, random passwords.
- Improves resistance to phishing attacks.

Password Manager Approaches

1. **Password Wallet**
   - Manages existing passwords, automatically selecting the password needed based on prior association with the domain of use.
2. **Derived Passwords**
   - Application-specific or site-specific passwords derived from a master password plus other information such as the target domain.
     - Example: PwdHash and Password Multiplier tools.

Graphical Passwords

Types

1. **Pure Recall**: User reconstructs a pattern.
2. **Cued Recall**: User is aided by a graphical cue.
3. **Recognition Schemes**: User recognizes previously seen images.

Advantages

– Easier to remember.
– Potentially more secure than text passwords.

Example

Android swipe patterns and click-based graphical passwords.

Account Recovery and Secret Questions

Recovery Methods

1 **Recovery Passwords and Links**

– Temporary passwords or web page links sent to recovery email addresses.

2 **Codes Sent to Telecom Device**

– One-time recovery codes sent to pre-registered phone numbers.

3 **Question-Based Recovery**

– Secret questions or challenge questions answered to reset passwords.

Usability and Security Aspects

- Recovery by challenge questions often fails due to non-unique answers, changes over time, and ease of guessing.

One-Time Passwords and Hardware Tokens

- **OTP Generators**
  - Use a **secret** and a time-varying parameter to generate a **one-time password**.
- **Hardware Tokens**
  - Physical devices that generate authentication tokens.

OTPs Received by Mobile

- OTPs can be sent to users via **SMS**, serving as a second factor in authentication.
  ! SIM Swap Attack !
  - Attackers use social engineering to trick providers into transferring a victim's number to a new SIM.
  - This allows attackers to receive OTPs intended for the victim.

OTPs from Lamport Hash Chains

1. **Initialization**:
   - Start with a random secret (seed) (w).
   - Generate a sequence of OTPs using a one-way hash function (H).

$$h_0 = H^{100}(w)$$

1. **Usage**:
   - Each session uses a decrementing index (i).
   - Server stores the initial hash (h_0) for verification.
   - For each authentication, use (h_{100-i}).

session *i*	user *A*		server *B*
i = 0 (setup)	secret w, $h_0 = H^{100}(w)$ ⋮ $h_{75} = H^{25}(w)$		Setup value: $v \leftarrow h_0$ ⋮ … last update: $v \leftarrow h_{75}$
i = 76	Compute $h_{76} = H^{24}(w)$	*A*, $x = h_{76}$ →	Receive *x;* if $H(x) = v$ allow (else deny). Update v ← x
i = 77	Compute $h_{77} = H^{23}(w)$	*A*, $x = h_{77}$ →	Receive *x;* if $H(x) = v$ allow (else deny). Update v ← x

Lamport Hash Chain for Session i = 76

- **Setup**: Store $(v \leftarrow h_0)$.
- **For Session i = 76**:
  - $(h_{24} = H^{24}(w))$
  - Update value: $(v \leftarrow h_{75})$

$$v = h_{100} \rightarrow h_{99} \rightarrow \ldots \rightarrow h_{76}$$

Passcode Generators

Commercial One-Time Password Generators

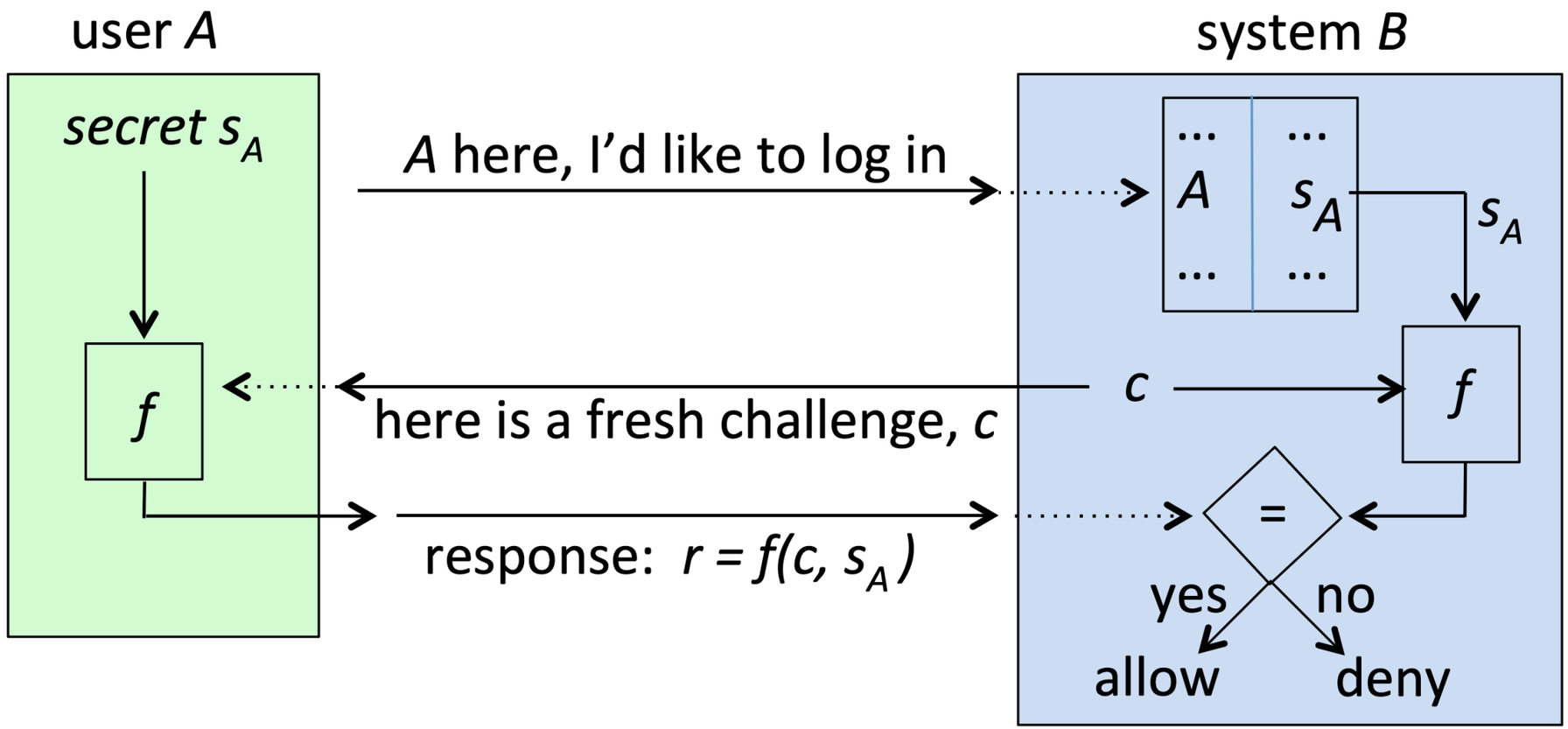
– Calculator-like devices or smartphone apps.

Photo by Jean-Luc Picard on Unsplash

Types of Challenges

1. **Explicit Challenge**:
   - An **8-digit string** sent by the system for the user to enter.
   - Device requires a keypad.

2. **Implicit Challenge**:
   - A time value, where the passcode remains constant for **one-minute windows**.
   - **Requires** a synchronized clock.


user A
secret s_A
f
A here, I'd like to log in
here is a fresh challenge, c
response: $r = f(c, s_A)$
system B
...
A
...
...
s_A
...
s_A
c
f
=
yes
no
allow
deny

1 what you know

5

2 what you have

6

7

multi-factor

where you are 4

8

what you are/do 3

Biometric Authentication

Types

– **Physical Biometrics**: Fingerprints, facial recognition, iris recognition.
– **Behavioral Biometrics**: Voice authentication, gait, typing rhythm.

Advantages

– No need to remember passwords.
– Generally perceived as more convenient.

Challenges

– Biometrics are not secrets; can be captured and reused.
– Difficulty in changing biometric data if compromised.

Modality	Type	Notes
fingerprints	P	common on laptops and smartphones
facial recognition	P	used by some smartphones
iris recognition	P	the part of the eye that a contact lens covers
hand geometry	P	hand length and size, also shape of fingers and palm
retinal scan	P	based on patterns of retinal blood vessels
voice authentication	Mixed	physical-behavioral mix
gait	B	characteristics related to walking
typing rhythm	B	keystroke patterns and timing
mouse patterns	B	also scrolling, swipe patterns on touchscreen devices

Enrollment and Verification

Process

- Enrollment: Sample biometric measurements to build a reference template.
- Verification: Compare freshly taken sample to the template.

Example

iPhone fingerprint and face recognition.

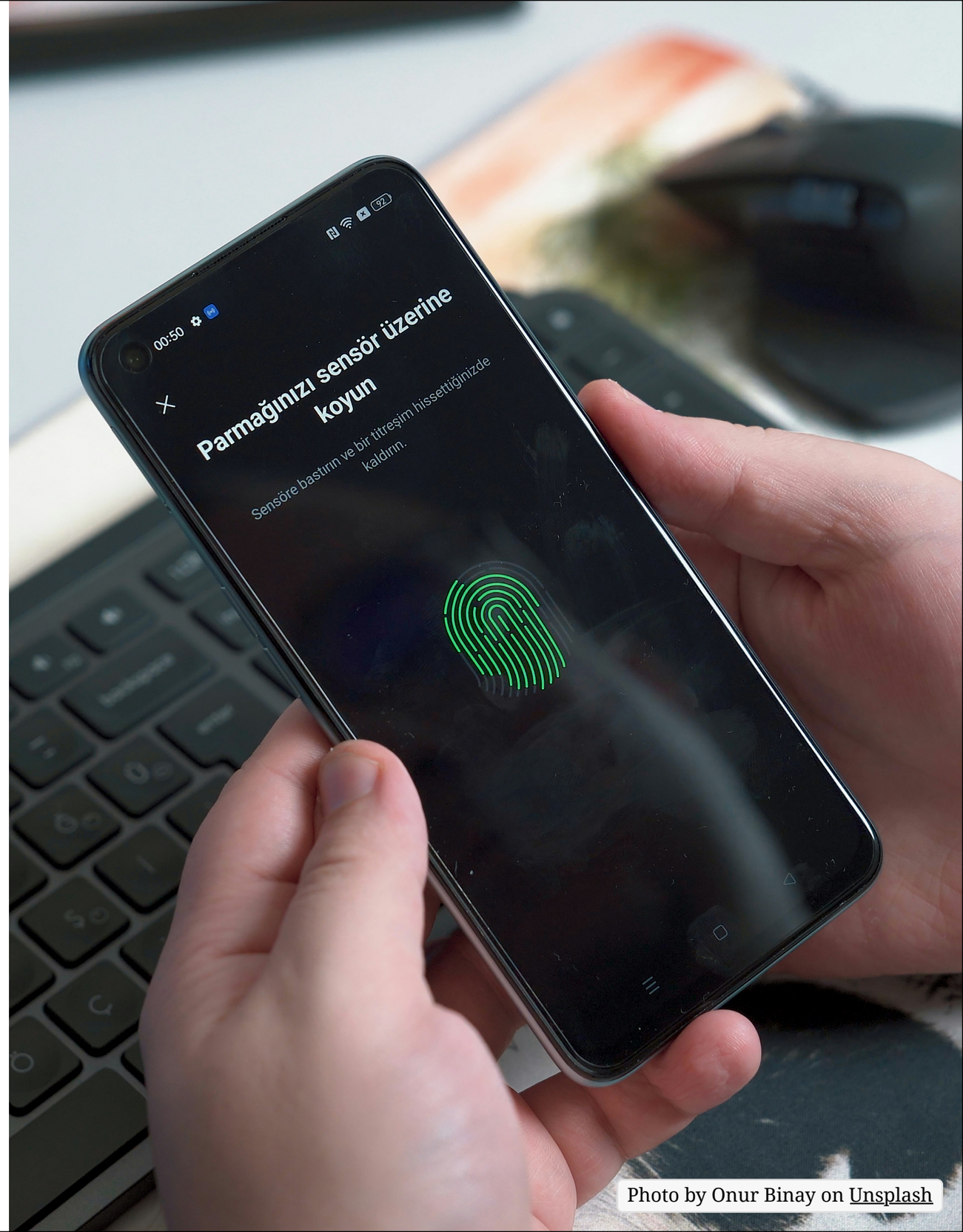


Photo by Onur Binay on Unsplash

Biometrics as Non-Secrets

- **Non-Secret Nature**: Biometrics are **easily** obtainable and not secret.
- **Trusted Input Channel**: Ensures that the biometric sample is from the correct individual.

  Example
- Fingerprints left on surfaces can be copied.
- Faces are easily visible and can be photographed.

Failure to Enroll (FTE) and Failure to Capture (FTC)

- **Failure to Enroll (FTE)**: Inability to **register** a biometric template.
- **Failure to Capture (FTC)**: Inability to capture a **usable sample** during verification.

Factors Affecting FTE and FTC

- Device limitations
  - User's physical condition (e.g., dry skin for fingerprint reading)
  - Environmental factors (e.g., lighting for facial recognition)

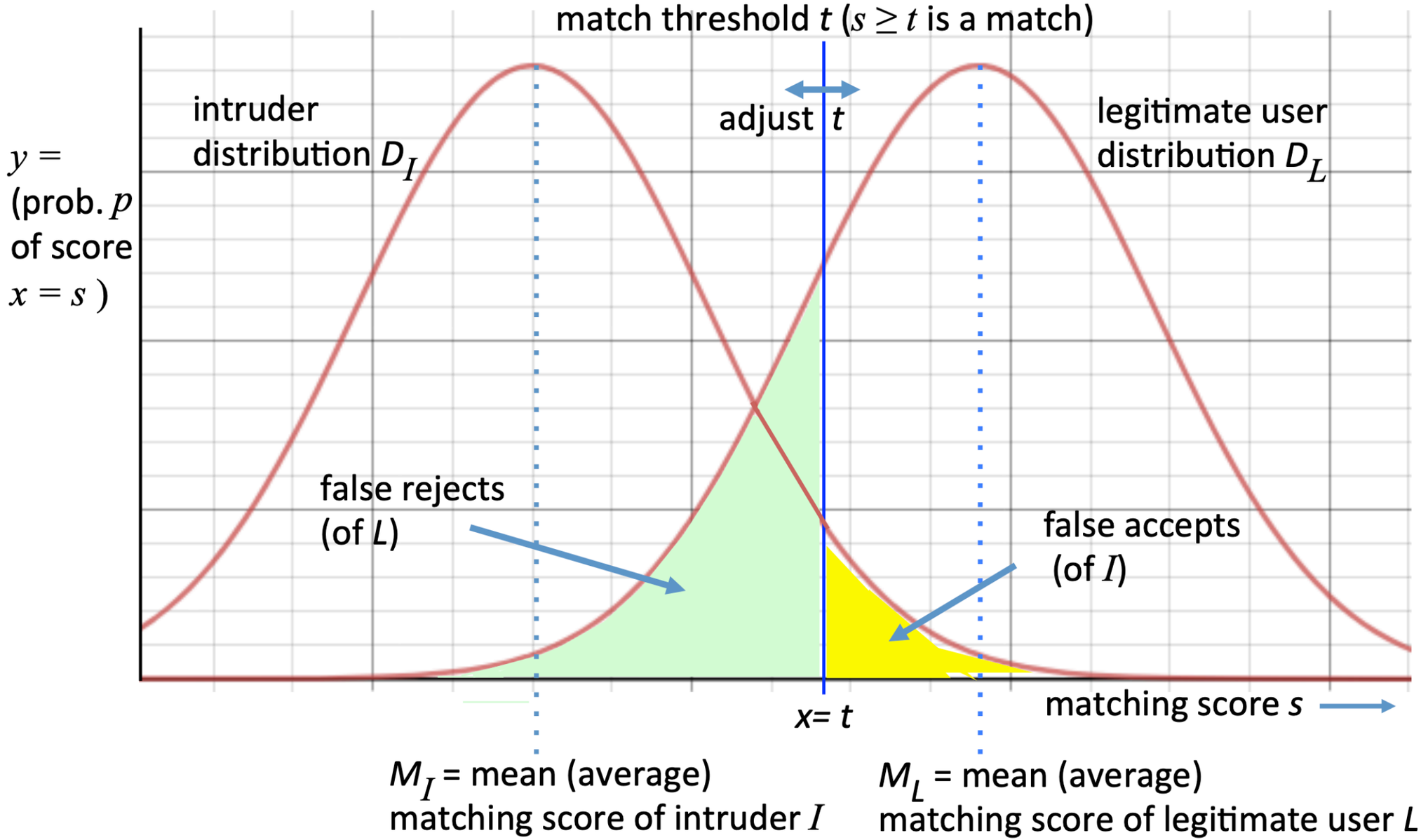
False Rejects (FRR) and False Accepts (FAR)

Definitions

– **False Reject Rate (FRR)**: Probability that a legitimate user is **incorrectly rejected**.
– **False Accept Rate (FAR)**: Probability that an imposter is **incorrectly accepted**.

Trade-Offs

- Stricter thresholds increase `FRR` but reduce `FAR`.
- Looser thresholds decrease `FRR` but increase `FAR`.

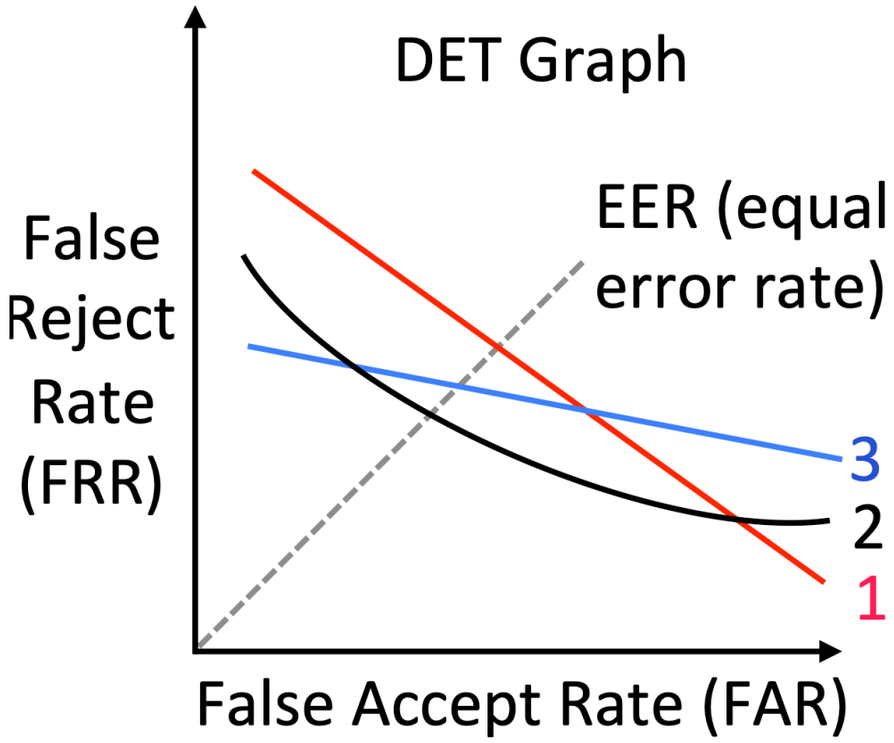

match threshold t ($s \geq t$ is a match)
$y =$ (prob. p of score $x = s$)
intruder distribution D_I
adjust t
legitimate user distribution D_L
false rejects (of L)
false accepts (of I)
$x = t$
matching score s
M_I = mean (average) matching score of intruder I
M_L = mean (average) matching score of legitimate user L

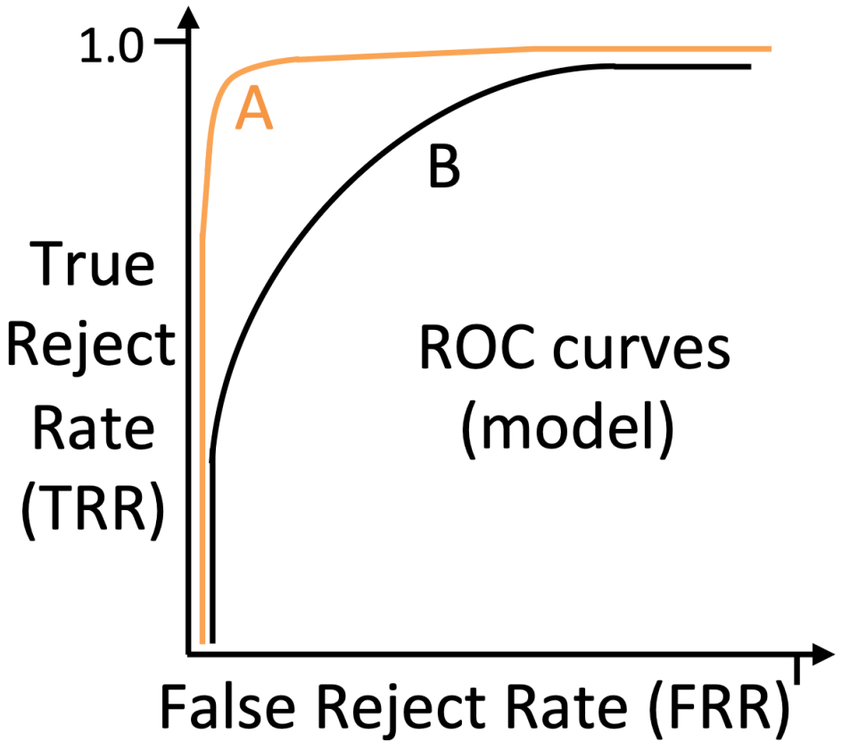
Equal Error Rate (EER)

- **Equal Error Rate (EER)**: Point where **FAR equals FRR**.
- EER is used for simplified single-point comparisons of biometric systems.

Graph: DET and ROC Curves

- **DET Curve**: Plots FAR against FRR.
- **ROC Curve**: Plots True Positive Rate (TPR) against False Positive Rate (FPR).


DET Graph
False Reject Rate (FRR)
EER (equal error rate)
3
2
1
False Accept Rate (FAR)


1.0
A
B
True Reject Rate (TRR)
ROC curves (model)
False Reject Rate (FRR)

Standard Criteria for Biometrics Evaluation

1. **Universality**: Availability of the characteristic in all users.
2. **Distinguishability**: Differences between users' characteristics.
3. **Invariance**: Stability of characteristics over time.
4. **Ease of Sampling**: Simplicity of obtaining samples.
5. **Accuracy**: Measured by FAR, FRR, EER, FTE-rate, FTC-rate.
6. **Cost**: Time, storage, hardware/software costs.
7. **User Acceptance**: Willingness of users to adopt the system.
8. **Attack Resistance**: Ability to withstand impersonation or spoofing attacks.